Deep Learning and Computation

The deep learning story — Lecture 18 (Chapter 10, Part 1)

Amir Farbin

Part 1 — The deep learning story

Where we left off

- Classical machine learning learns **where to draw the boundary**, on features someone chose
- Deep learning learns **the features too**: from pixels, sound or particle hits to an answer
- Today: a first, intuitive picture of what these networks do. The pieces come later

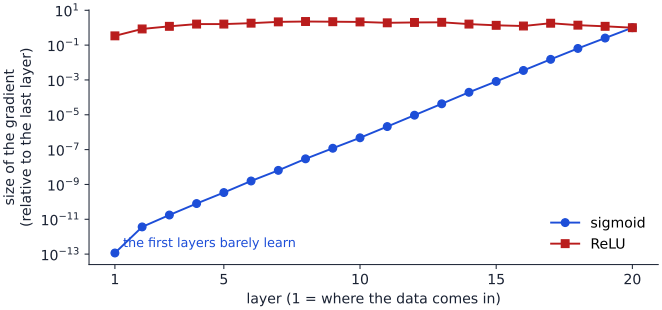
An old idea

- **1943:** a mathematical neuron: weigh the inputs, and fire past a threshold
- **1958:** the perceptron, a machine that **learned** its weights, with motor-turned resistors
- **1989:** universal approximation: one hidden layer can approximate **any** continuous function
  - but it says nothing about **how many** neurons, or how to **find** the weights

Deeper is more efficient

- Universality says one wide layer is **enough**. It doesn't say it's **efficient**
- A deep network represents many functions with **far fewer** neurons than a shallow one
- Like computing in steps: each layer reuses what the one before it computed

But deep networks wouldn't train



- The gradient for an early layer is a **product** of one factor per layer above it (the chain rule)
- With sigmoids each factor is small, so the **first layers**, where the data comes in, barely learn

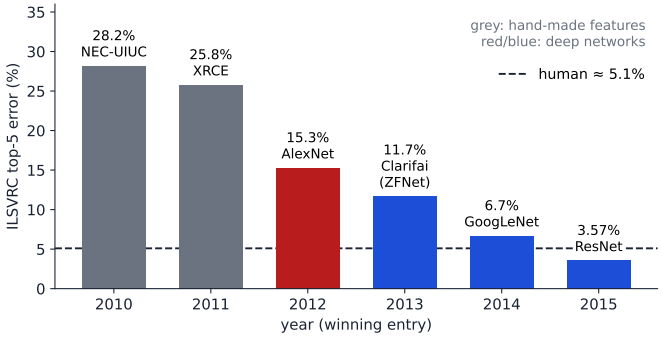
The quiet years

- **1969:** a book proves the limits of a single layer. Interest and funding dry up
- **1991:** the vanishing gradient is identified. Deep networks are set aside
- The 1990s: AI turns to **symbolic** systems, logic and rules, which don't get very far either
- A few keep going: LeCun's networks read zip codes, then cheques

What changed

- **A simpler activation:** ReLU, “a line and a line”. Its slope is 1 wherever it’s on, so gradients don’t shrink
- **Better training:** stochastic gradient descent on mini-batches, better starting weights, normalization, shortcut connections
- **Big data:** the internet; millions of labeled images
- **GPUs:** built for games, they do exactly the arithmetic training needs

2012: the renaissance begins

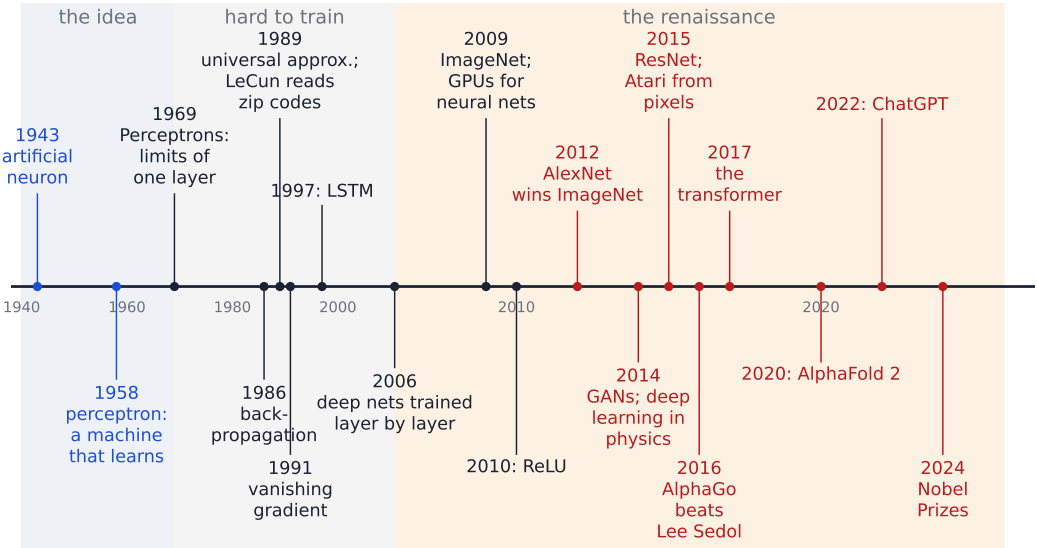


- **ImageNet:** about a million labeled images, 1,000 categories. Labels by paid crowd workers
- Hand-built features: ~26% error. **AlexNet**, a deep network on two gaming GPUs: 15%. By 2015, better than a person

And then, everything

- **2013–15:** Atari games learned from the pixels and the score alone
- **2014:** deep networks in particle physics, on the SUSY dataset you've used
- **2016:** AlphaGo beats a world champion at Go; players start learning from it
- **2020:** AlphaFold predicts protein structures
- **2022:** ChatGPT
- **2024:** Nobel Prizes in physics (Hopfield, Hinton) and chemistry (Hassabis, Jumper)

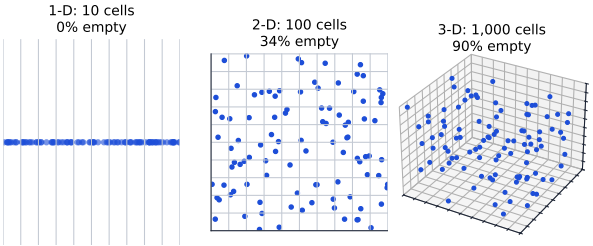
The timeline



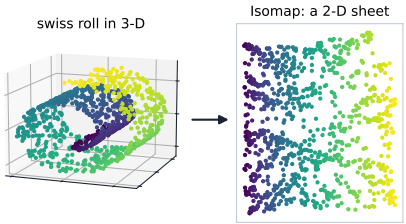
Learning a representation

- Your brain keeps a **model** of this room from glimpses, and acts on the model
- A network maps raw data into a **new space** where the task is easy
- The perfect function exists, because brains compute it. We **approximate** it from examples

Most of space is empty, and data lives on a thin sheet



the same N = 100 points, 10 bins per axis



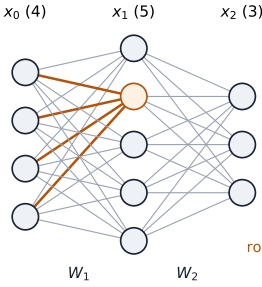
colour = position along the roll

- More dimensions, more boxes: almost all of them empty (the **curse of dimensionality**)
- A 100×100 photo is one point in 10,000 dimensions, but all photos of one face lie on a thin, curved **sheet**. Walking on it is **turning your head**, or **smiling**
- Learning features is learning the **coordinates** on that sheet

Why deep: build on what's below

- Pixels → edges → parts → objects. Characters → words → sentences
- One layer **builds** a feature, the next **combines** it
- Low layers are **shared** by everything; high layers are **specific**

The building block



inputs (4 columns)

outputs (5 rows)

w_{11}	w_{12}	w_{13}	w_{14}
w_{21}	w_{22}	w_{23}	w_{24}
w_{31}	w_{32}	w_{33}	w_{34}
w_{41}	w_{42}	w_{43}	w_{44}
w_{51}	w_{52}	w_{53}	w_{54}

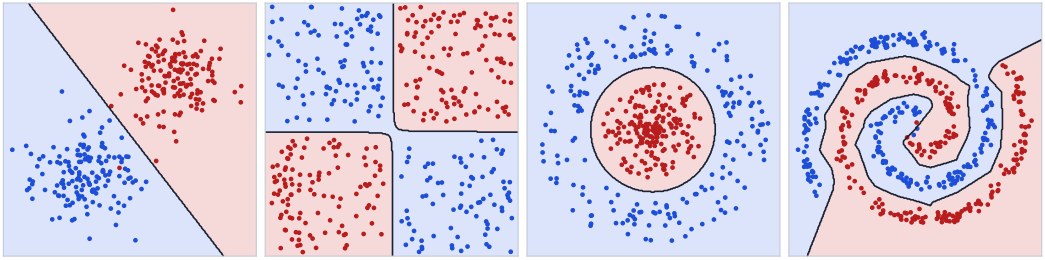
W_1: 5×4

row 2 of W_1: the weights into hidden unit 2

$$x_i = f(W_i x_{i-1} + b_i)$$

- A **layer**: a linear model, then a nonlinearity. A **network**: layers stacked
- Without the nonlinearity, a stack of layers is still **one** linear model

Feature engineering, or feature learning



linear
inputs x_1, x_2
training accuracy 100%

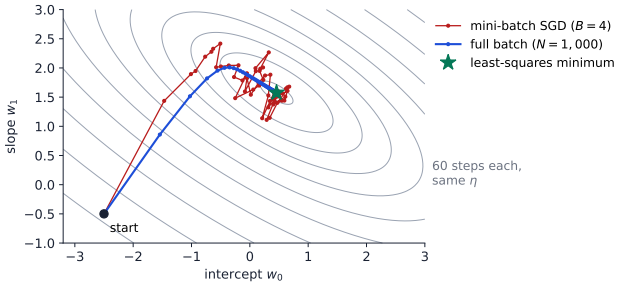
XOR
add $x_1 x_2$
training accuracy 100%

circle
add $x_1^2 + x_2^2$
training accuracy 100%

spiral
2 hidden layers of 16 (ReLU)
training accuracy 99%

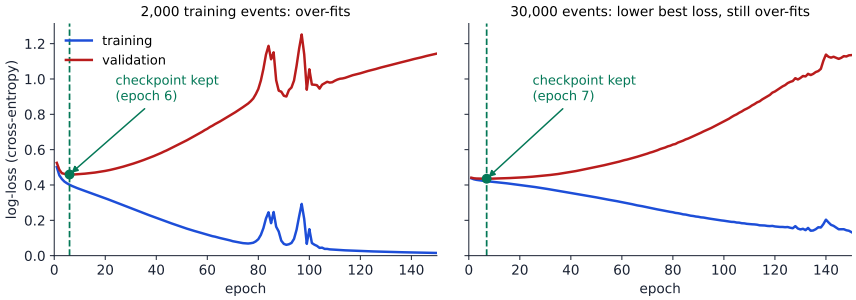
- XOR: multiply the two inputs and one neuron is enough. The circle: all you need is the **radius**
- The spiral: let a network **find** the features

Training is walking downhill



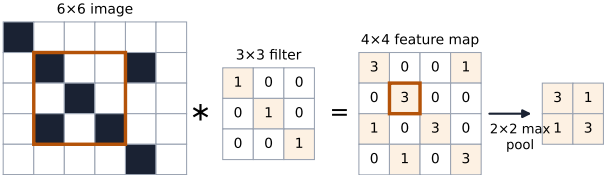
- Height = how wrong you are. Position = the parameters. Step **against** the slope
- Estimate the slope on a **mini-batch**: many cheap, noisy steps, and the noise helps
- Scale your inputs, and let the framework compute the slope for you

It can memorize

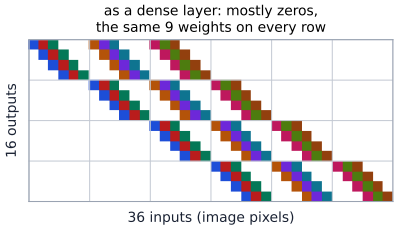


- A big enough network can learn every training example by heart
- Keep data it never trains on, and watch the two curves separate
- **Dropout**: knock out random units while training, so it can't lean on any one

A filter is a detector



highlighted window: sum of image × filter = 3 (the diagonal it matches)



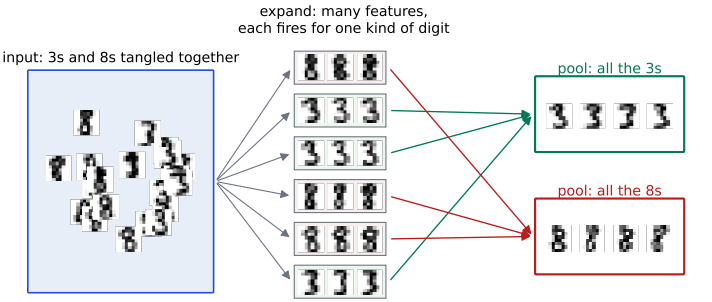
colour = which of the 9 filter weights

- Slide a small patch over the image, multiply and add: a big number **where the pattern is**
- The identity-matrix patch lights up on **diagonal lines**: a diagonal-line detector
- One filter per pattern gives one **feature map** per pattern

Stack, shrink, repeat

- Shrink the maps, then look for patterns **in the maps**: lines make curves, curves make parts
- Repeat until the image is one pixel wide, with many features
- Local and general $\rightarrow$ **global and specific**: the chair detector doesn't say where the chair is
- I don't build the detectors. I set it up as "cat or not", and it **learns** every filter

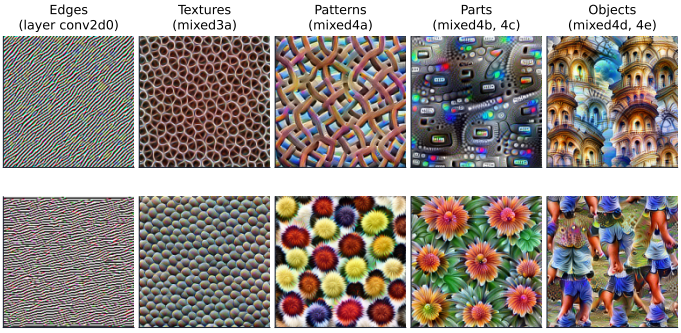
Expand, then pool



in the expanded space a simple cut separates them; pooling keeps what matters and forgets the style

- Lift the data into **more** dimensions, where a simple cut separates it; then **pool** what's alike

What the layers learn

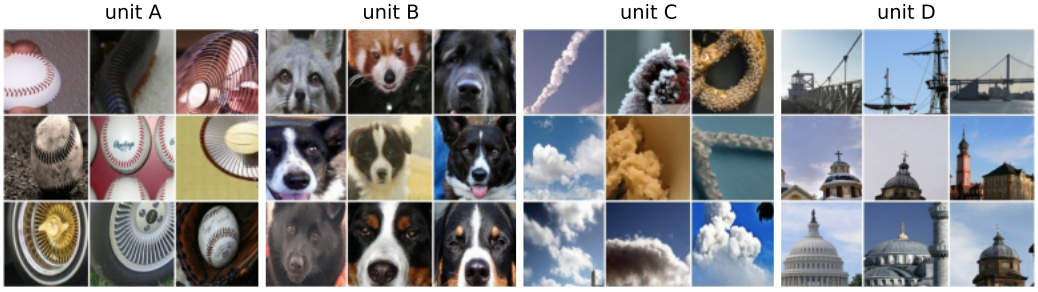


early layers → late layers: what one unit in each layer responds to most strongly (optimized images)

- Early layers: **edges and textures**, the same whatever you train on. Later: parts, then objects

Olah, Mordvintsev & Schubert, "Feature Visualization", Distill (2017), CC-BY 4.0

Which photos light up a unit

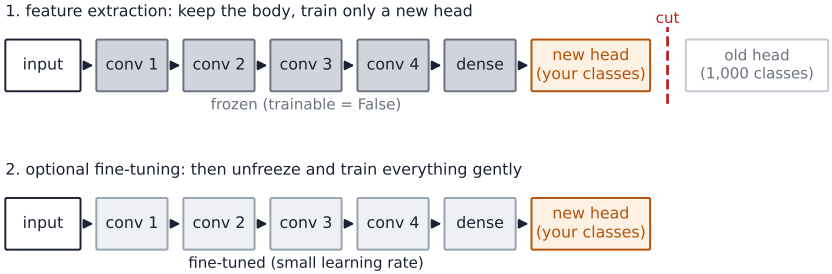


for each of four units deep in the network: the 9 photos that activate it most

- Run many photos through, and keep the ones that activate one unit most
- Deep in the network, units have become detectors of **things**: dog faces, clouds, domes

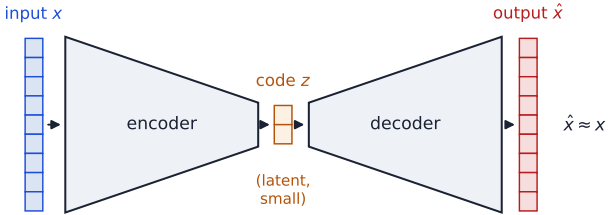
Olah, Mordvintsev & Schubert, "Feature Visualization", Distill (2017), CC-BY 4.0

Features belong to the domain



- What a network learns about images is about **images**, not just its task
- Chop off the last layer, put on yours, train only that. Then fine-tune everything if you can

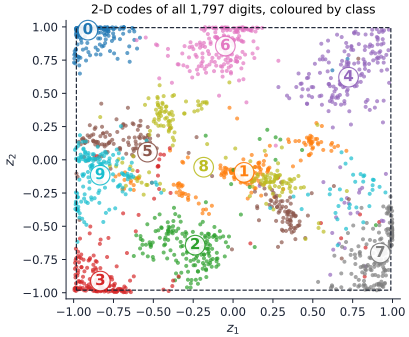
Learning without labels: the autoencoder

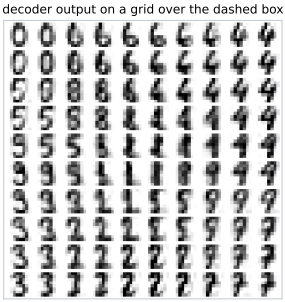


loss $= \|x - \hat{x}\|^2$ (the target is the input itself: no labels, self-supervised)

- Squeeze the input through a **bottleneck** and reproduce it. No labels needed
- The small **code** in the middle, the **latent space**, keeps what's unique about each example
- Change the target: noisy in, clean out; small in, large out

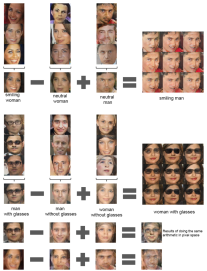
The latent space is a map

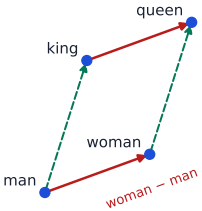




- Squeeze digits to **two** numbers: similar digits land close together
- Pick a point where the sevens live and decode it: a new seven. Close codes mean similar things

Directions mean something





king − man + woman ≈ queen

the same offset (a “gender” direction)
links both pairs

≈: queen is the nearest word vector
after excluding king, man and woman

word2vec, Mikolov et al. (2013); schematic 2-D picture

- Faces with glasses − faces without = a “glasses” direction. Add it and the glasses appear; the same for a **smile**
- Words: king − queen = man − woman, so king − man + woman ≈ queen

Faces: Radford, Metz & Chintala, arXiv:1511.06434, Fig. 7

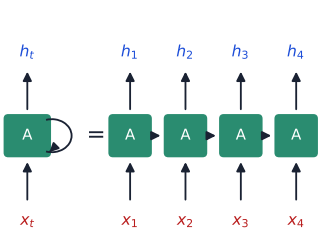
Making new data



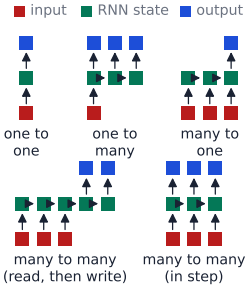
auto-regressive	**VAE**	**GAN**	**diffusion**	**normalizing flow**
one piece at a time, each given the ones before: $\prod_i p(x_i \mid x_{<i})$	encode to z, decode back; generate from $z \sim N(0, I)$	generator vs. discriminator: a two-player game	learn to remove noise step by step; start from pure noise	invertible map from $N(0, I)$; exact $p(x)$

- Learn where real data lives, then **sample** there: there's always randomness
- Force the codes into a standard normal and draw new ones; or train a maker against a detector; or learn to **undo noise**

Sequences: read, remember, decide

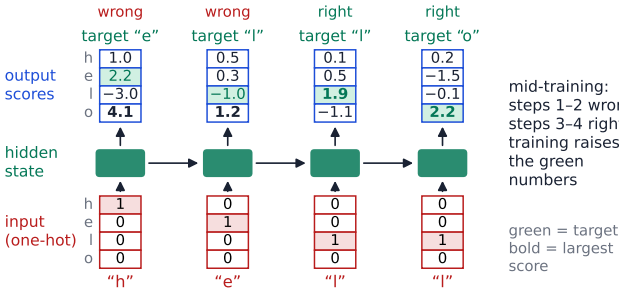


same cell A, same weights, every step



- Text and audio come in any length. Read one step at a time
- A second output, fed back in, is the **memory**
- Many-to-one: a review, good or bad. Many-to-many: translation

Predict the next token

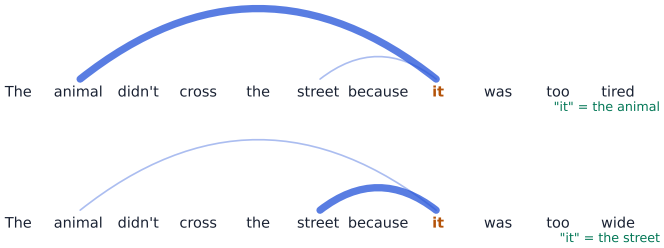


- Inputs “h e l l”, targets “e l l o”: the **labels are free**
- Do it on all the text there is: that’s how large language models are **pre-trained**

Why transformers won

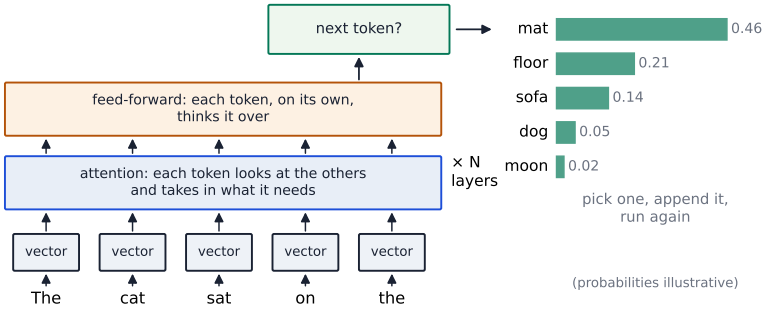
- A recurrent network can't compute step 5 before step 4: training is **sequential**
- The transformer drops the recurrence, so a whole sequence trains **in parallel**
- Then scale: bigger models and more data have kept getting better

Attention: which words matter to me?



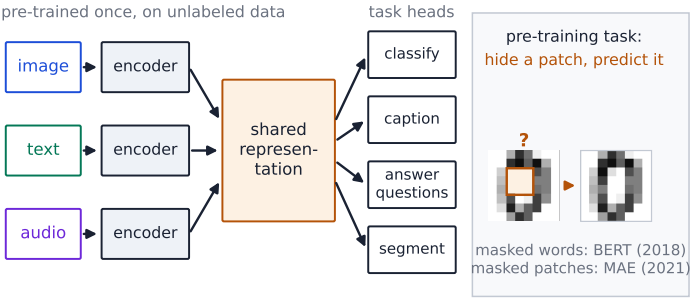
- Each word **compares** itself with every other word, turns the scores into weights, and takes a weighted mix
- What "it" means depends on the rest of the sentence: attention finds the words that decide it

A transformer



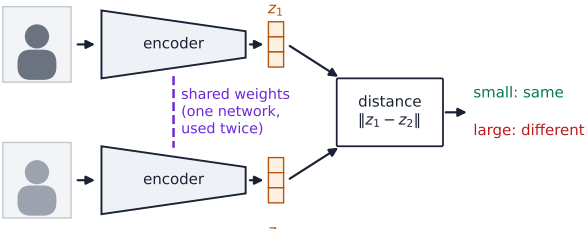
- **Attention:** tokens exchange information. **Feed-forward:** each token thinks it over. Repeat
- At the top: probabilities for the next token. Pick one, append it, run again

Foundation models



- Pre-train on huge **unlabeled** data: hide part of the input, predict it
- One shared representation, a small head per task. They do tasks nobody trained them for

Lego blocks



- Images → convolutions. Sequences → recurrent nets or transformers. Irregular data → graphs
- The badge reader: **one** encoder on both photos, then "same person?"

In my field

- 2014: deep networks on low-level detector data match decades of hand-built features
- Graph networks on calorimeters; transformers on sets of particles
- Autoencoders in the trigger, to keep events nobody thought to look for
- Networks made **blind** to mass, so a bump is a particle, not something the network made

What comes next

- **scikit-learn:** classical machine learning in a few lines (Lecture 18's notebook)
- **The software:** how frameworks compute and differentiate, and Keras (Lecture 20)
- **The pieces:** layers, activations, costs, optimizers, over-fitting, and the architectures